

Politechnika Świętokrzyska	
Wydział Elektrotechniki, Automatyki i Informatyki	
Przetwarzanie odporne na błędy	Temat projektu: 4 - CRC
Skład zespołu: • Przemysław Kałuziński • Michał Kaczor • Jakub Kuśmierczyk • Bartłomiej Wydrzycki	Data wykonania dokumentacji: 04.11.2025r.

1. Spis treści

1.	Spis treści	1
1.	Package – pob.pob_project.crc	7
1.1.	Class – CRCUtil	7
1.1.1.	public String computeCRC(String message, String polynomial)	7
1.1.2.	private String toBinary(String message)	7
1.1.3.	private char xorBit(char a, char b)	7
1.1.4.	public String appendCRC(String message, String polynomial)	7
1.1.5.	public boolean validateCRC(Packet packet, String polynomial)	7
1.1.6.	public String extractMessage(String dataWithCrc, String polynomial)	8
1.1.7.	public String extractCRC(String dataWithCrc, String polynomial)	8
1.1.8.	public String fromBinary(String bits)	8
2.	Package – pob.pob_project.error	9
2.1.	Class – ErrorInjector	9
2.1.1.	public void applyError(Node node, ErrorType type)	9
2.2.	Enum – ErrorType	9
2.2.1.	BIT_FLIP	9
2.2.2.	PACKET_DROP	9
2.2.3.	NODE_FREEZE	9
2.2.4.	DELAY	9
2.2.5.	CRC_FAILURE	9
2.3.	Class – Fault	9
2.3.1.	Field Details	9
	• type	9

• active	10
2.3.2. public Fault(ErrorType type)	10
2.3.3. public void deactivate()	10
2.3.4. public ErrorType getType()	10
2.3.5. public boolean isActive()	10
3. Package – pob.pob_project.gui	11
3.1. Class – ControlPanel	11
3.1.1. Field Details	11
• controller	11
• graphPanel	11
3.1.2. public ControlPanel(SimulationController controller, GraphPanel graphPanel).....	11
3.1.3. private void showAlert(String title, String message).....	11
3.2. Class – GraphPanel	11
3.2.1. Field Details	12
• WIDTH	12
• HEIGHT	12
• controller	12
• nodeCircles	12
• nodeLabels	12
• errorLabels	12
• connections	13
• statsLabels	13
3.2.2. public GraphPanel(SimulationController controller).....	13
3.2.3. private void drawGraph()	13
3.2.4. public void updateNodeStats(Node node)	13
3.2.5. public void animateTransmission(Node from, Node to, long durationMs) 13	
3.2.6. public void updateNodeStatus(Node node)	13
3.3. Class – LogPanel	13
3.3.1. Field Details	14
• logFlow	14

3.3.2.	public LogPanel()	14
3.3.3.	public static void appendLog(String message)	14
3.3.4.	private static String extractTime(String msg)	14
3.3.5.	private static String extractNodeId1(String msg)	14
3.3.6.	private static String extractRest1(String msg)	14
3.3.7.	private static String extractNodeId2(String msg)	14
3.3.8.	private static String extractBits	15
3.3.9.	private static String extractRest2(String msg)	15
3.3.10.	public static void clearLog()	15
3.4.	Class – MainApp	15
3.4.1.	Field Details	15
•	controller	15
3.4.2.	public void start(Stage stage)	15
3.4.3.	public static void main(String[] args)	15
3.5.	Class – MainWindow	16
3.5.1.	Field Details	16
•	graphPanel	16
•	controlPanel	16
•	logPanel	16
3.5.2.	public MainWindow(SimulationController controller)	16
3.5.3.	public GraphPanel getGraphPanel()	16
4.	Package – pob.pob_project.network	17
4.1.	Class – Connection	17
4.1.1.	Field Details	17
•	nodeA	17
•	nodeB	17
4.1.2.	public Connection(Node a, Node b)	17
4.1.3.	public Node getNodeA()	17
4.1.4.	public Node getNodeB()	17
4.2.	Class – NetworkGraph	17
4.2.1.	Field Details	17

• nodes	17
• connections	18
4.2.2. public void addNode(Node n)	18
4.2.3. public void addConnection(Node a, Node b)	18
4.2.4. public List<Node> getNodes()	18
4.2.5. public List<Connection> getConnections().....	18
4.2.6. public List<Node> getNeighbors(Node n)	18
4.3. Class – Node	18
4.3.1. Field Details	19
• id	19
• isActive	19
• incomingQueue	19
• neighbors	19
• currentFault	19
• polynomial	19
• crcUtil	19
• nodeThread	20
• controller	20
• lastSentPackets	20
• sentCount	20
• receivedCount	20
• errorCount	20
4.3.2. public Node(int id, SimulationController controller)	20
4.3.3. public void addNeighbor(Node n).....	20
4.3.4. public void start().....	21
4.3.5. public void sendData(Node target, String message)	21
4.3.6. public void receivePacket(Packet packet)	21
4.3.7. public void run().....	21
4.3.8. private Node findNeighborById(int id).....	21
4.3.9. private String flipRandomBit(String data, String polynomial)	21
4.3.10. public void injectFault(ErrorType type).....	21

4.3.11.	public void repairFault()	21
4.3.12.	public void setCrcPolynomial(String polynomial)	21
4.3.13.	private void updateStatsInUI().....	21
4.3.14.	public int getId().....	21
4.3.15.	public BlockingQueue<Packet> getQueue().....	22
4.3.16.	public boolean isActive()	22
4.3.17.	public List<Node> getNeighbors()	22
4.3.18.	public Fault getCurrentFault().....	22
4.3.19.	public String getCurrentErrorText()	22
4.3.20.	public int getErrorCount()	22
4.3.21.	public int getSentCount()	22
4.3.22.	public int getReceivedCount()	22
4.4.	Class – Packet	22
4.4.1.	Field Details	22
•	data	22
•	sourceId	23
•	destinationId	23
•	isDelayed	23
•	delay	23
•	isAck	23
•	ackPositive	23
•	random	23
4.4.2.	public Packet(String data, int sourceId, int destinationId)	23
4.4.3.	public static Packet createAckPacket(int sourceId, int destinationId, boolean ackPositive).....	24
4.4.4.	public String getData()	24
4.4.5.	public int getSourceId()	24
4.4.6.	public int getDestinationId()	24
4.4.7.	public boolean isDelayed()	24
4.4.8.	public void setIsDelayed(boolean isDelayed).....	24
4.4.9.	public boolean isAck().....	24
4.4.10.	public boolean isAckPositive()	24

4.4.11.	public void setAck(boolean ack)	24
4.4.12.	public void setAckPositive(boolean ackPositive)	25
4.4.13.	public void setDelay(int delay)	25
4.4.14.	public int getDelay()	25
5.	Package – pob.pob_project.simulation.....	26
5.1.	Class - Logger	26
5.1.1.	Field Details	26
•	sdf	26
5.1.2.	public static synchronized void log(String message)	26
5.2.	Class – SimulationController	26
5.2.1.	Field Details	26
•	graph	26
•	errorInjector	26
•	crcPolynomial	26
•	graphPanel	27
5.2.2.	public SimulationController()	27
5.2.3.	public void initializeNetwork().....	27
5.2.4.	public void startSimulation().....	27
5.2.5.	public boolean checkCrcPolynomial(String polynomial).....	27
5.2.6.	public boolean checkNeighbors(Node src, Node dst).....	27
5.2.7.	public void injectError(Node node, ErrorType type)	27
5.2.8.	public void repairNode(Node node)	27
5.2.9.	public NetworkGraph getGraph()	27
5.2.10.	public String getCrcPolynomial().....	28
5.2.11.	public void setCrcPolynomial(String newPoly)	28
5.2.12.	public void setGraphPanel(GraphPanel panel)	28
5.2.13.	public GraphPanel getGraphPanel()	28

1. Package – pob.pob_project.crc

1.1. Class – CRCUtil

Klasa obliczająca kod CRC metodą dzielenia modulo-2 (XOR). Działa dla dowolnego wielomianu binarnego wprowadzonego przez użytkownika.

1.1.1. ***public String computeCRC(String message, String polynomial)***

Oblicza CRC dla dowolnego tekstu.

Parameters:

message - Tekst do zabezpieczenia.

polynomial - Dowolny wielomian.

Returns:

CRC w postaci binarnej.

1.1.2. ***private String toBinary(String message)***

Pomocnicza metoda, która używa StandardCharsets.UTF_8. Tworzy prawdziwe bajty, a nie pojedyncze znaki, aby korzystać z polskich znaków.

Parameters:

message - Wiadomość do przekształcenia.

Returns:

Wiadomość w postaci binarnej.

1.1.3. ***private char xorBit(char a, char b)***

Wykonuje xor na bitach.

Parameters:

a - Pierwszy bit.

b - Drugi bit.

Returns:

Wynik działania xor.

1.1.4. ***public String appendCRC(String message, String polynomial)***

Łączy wiadomość z wygenerowanym CRC.

Parameters:

message - Wiadomość do połączenia z CRC.

polynomial - Wielomian do zabezpieczenia wiadomości.

Returns:

Ciąg bitów z CRC.

1.1.5. ***public boolean validateCRC(Packet packet, String polynomial)***

Sprawdza poprawność CRC, jeśli wynik składa się z zer, zwraca true.

Parameters:

packet - Pakiet z danymi.

polynomial - Wielomian, przy pomocy którego dane są zabezpieczone.

Returns:

Wartość true lub false.

1.1.6. *public String extractMessage(String dataWithCrc, String polynomial)*

Wyodrębnia wiadomość z ciągu bitów zawierającego CRC.

Parameters:

dataWithCrc - Ciąg bitów wiadomości + CRC.

polynomial - Wielomian, przy pomocy którego dane są zabezpieczone.

Returns:

Część wiadomości (bity bez CRC).

1.1.7. *public String extractCRC(String dataWithCrc, String polynomial)*

Wyodrębnia CRC z ciągu bitów wiadomości + CRC.

Parameters:

dataWithCrc - Ciąg bitów wiadomości + CRC.

polynomial - Wielomian, przy pomocy którego dane są zabezpieczone.

Returns:

część CRC (reszta z dzielenia).

1.1.8. *public String fromBinary(String bits)*

Zamienia ciąg bitów na String.

Parameters:

bits - Ciąg bitów.

Returns:

String z bitów.

2. Package – pob.pob_project.error

2.1. Class – ErrorInjector

Klasa odpowiedzialna za wstrzykiwanie błędów (usterki) do węzłów sieci. Umożliwia symulację różnych typów błędów, z typu wyliczeniowego ([ErrorType](#)). Klasa ta jest wykorzystywana przez kontroler symulacji do testowania zachowania sieci w warunkach błędów.

2.1.1. **public void applyError(Node node, ErrorType type)**

Wstrzykuje wybrany typ błędu do podanego węzła.

Parameters:

node - Węzeł, do którego ma zostać wprowadzony błąd.

type - Typ błędu, który ma zostać wstrzyknięty.

2.2. Enum – ErrorType

Typy błędów, które mogą wystąpić w węzłach sieci podczas symulacji. Enum definiuje różne scenariusze awarii, które można wstrzyknąć.

2.2.1. **BIT_FLIP**

Błąd bitowy - losowa zmiana pojedynczego bitu w danych pakietu.

2.2.2. **PACKET_DROP**

Utrata pakietu - pakiet nie zostaje dostarczony do celu.

2.2.3. **NODE_FREEZE**

Zamrożenie węzła - węzeł staje się nieaktywny i nie wysyła ani nie odbiera danych.

2.2.4. **DELAY**

Opóźnienie transmisji - pakiet jest dostarczany z losowym opóźnieniem czasowym.

2.2.5. **CRC_FAILURE**

Awaria modułu CRC - węzeł generuje błąd podczas obliczania CRC.

2.3. Class – Fault

Reprezentuje usterkę (błąd) w węźle sieci. Klasa przechowuje informacje o typie błędu ([ErrorType](#)) oraz o tym, czy usterka jest aktualnie aktywna.

2.3.1. **Field Details**

- **type**

```
private ErrorType type
```

Typ błędu powiązany z tą usterką.

- **active**

private boolean active

Flaga określająca, czy błąd jest aktualnie aktywny.

2.3.2. *public Fault(ErrorType type)*

Tworzy nową usterkę danego typu i automatycznie ją aktywuje.

Parameters:

type - Typ błędu, który ma zostać wstrzyknięty.

2.3.3. *public void deactivate()*

Dezaktywuje usterkę, przywracając normalne działanie węzła.

2.3.4. *public ErrorType getType()*

Zwraca typ błędu przypisany do tej usterki.

Returns:

Typ błędu jako [ErrorType](#).

2.3.5. *public boolean isActive()*

Sprawdza, czy usterka jest aktualnie aktywna.

Returns:

Wartość true jeśli błąd jest aktywny, false w przeciwnym wypadku.

3. Package – *pob.pob_project.gui*

3.1. Class – *ControlPanel*

Panel sterowania symulacją sieci komputerowej. Umożliwia użytkownikowi:

- Wstrzykiwanie i naprawianie błędów w wybranych węzłach,
- Wysyłanie wiadomości między węzłami,
- Zmianę wielomianu CRC,
- Czyszczenie logów w panelu [LogPanel](#).

3.1.1. *Field Details*

- ***controller***

```
private SimulationController controller
```

Główny kontroler logiki symulacji.

- ***graphPanel***

```
private GraphPanel graphPanel
```

Panel graficzny przedstawiający węzły i połączenia między nimi.

3.1.2. ***public ControlPanel(SimulationController controller, GraphPanel graphPanel)***

Tworzy nowy panel sterowania dla danej symulacji.

Parameters:

controller - Kontroler symulacji, odpowiedzialny za logikę.

graphPanel - Panel graficzny przedstawiający sieć węzłów.

3.1.3. ***private void showAlert(String title, String message)***

Wyświetla komunikat ostrzegawczy w oknie dialogowym.

Parameters:

title - Tytuł okna ostrzeżenia.

message - Treść wiadomości do wyświetlenia.

3.2. Class – *GraphPanel*

Klasa odpowiedzialna za wizualizację grafu sieci w symulacji. Reprezentuje węzły, połączenia między nimi oraz wizualne efekty transmisji danych i błędów.

3.2.1. *Field Details*

- **WIDTH**

`private final int WIDTH`

Szerokość panelu graficznego w pikselach.

See Also:

- [Constant Field Values](#)

- **HEIGHT**

`private final int HEIGHT`

Wysokość panelu graficznego w pikselach.

See Also:

- [Constant Field Values](#)

- **controller**

`private SimulationController controller`

Kontroler zarządzający logiką i stanem symulacji.

- **nodeCircles**

`private Map<Node,javafx.scene.shape.Circle> nodeCircles`

Mapa powiązań węzłów z ich graficznymi reprezentacjami.

- **nodeLabels**

`private Map<Node,javafx.scene.text.Text> nodeLabels`

Mapa etykiet z identyfikatorami węzłów.

- **errorLabels**

`private Map<Node,javafx.scene.text.Text> errorLabels`

Mapa etykiet informujących o błędach węzłów.

- **connections**

```
private List<javafx.scene.shape.Line> connections
```

Lista linii reprezentujących połączenia pomiędzy węzłami.

- **statsLabels**

```
private Map<Node,javafx.scene.text.Text> statsLabels
```

Mapa etykiet wyświetlających statystyki węzłów.

3.2.2. **public GraphPanel(SimulationController controller)**

Konstruktor klasy GraphPanel. Inicjalizuje panel i rysuje graf w oparciu o dane z kontrolera symulacji.

Parameters:

controller - Kontroler symulacji zarządzający węzłami i połączeniami.

3.2.3. **private void drawGraph()**

Tworzy i rozmieszcza graficzne elementy reprezentujące węzły i połączenia grafu. Każdy węzeł ma etykietę, kolor, statystyki oraz reakcję na najechanie kursorem.

3.2.4. **public void updateNodeStats(Node node)**

Aktualizuje etykietę statystyk dla danego węzła (ilość wysłanych, odebranych i błędnych pakietów).

Parameters:

node - Węzeł, którego statystyki mają zostać zaktualizowane.

3.2.5. **public void animateTransmission(Node from, Node to, long durationMs)**

Animuje wizualny przesył danych pomiędzy dwoma węzłami. Na ekranie pojawia się mała kula symbolizująca pakiet, poruszająca się wzdłuż połączenia między węzłami.

Parameters:

from - Węzeł źródłowy (nadawca).

to - Węzeł docelowy (odbiorca).

durationMs - Czas trwania animacji w milisekundach.

3.2.6. **public void updateNodeStatus(Node node)**

Aktualizuje wygląd węzła (kolor i etykietę błędu) w zależności od jego aktualnego stanu. Zielony — węzeł działa poprawnie, czerwony — wystąpił błąd.

Parameters:

node - Węzeł, którego stan ma zostać zaktualizowany.

3.3. Class – LogPanel

Panel odpowiedzialny za wyświetlanie logów systemowych w interfejsie graficznym. Logi prezentowane są w postaci kolorowego tekstu. Klasa umożliwia także czyszczenie logów.

3.3.1. *Field Details*

• **logFlow**

```
private static javafx.scene.text.TextFlow logFlow
```

Główny obiekt przechowujący logi w formie kolorowego tekstu.

3.3.2. *public LogPanel()*

Konstruktor panelu logów. Inicjalizuje wygląd sekcji logowania oraz przewijalny obszar tekstu.

3.3.3. *public static void appendLog(String message)*

Dodaje nowy wpis do logów systemowych z wyróżnieniem kolorów i składników wiadomości. Wpis jest dzielony na części:

- znacznik czasu (zielony),
- identyfikatory komputerów (czerwony),
- dane binarne (żółto-zielony),
- pozostały tekst (beżowy).

Parameters:

message - Pełna treść logu do wyświetlenia.

3.3.4. *private static String extractTime(String msg)*

Wydobywa z logu fragment zawierający znacznik czasu w formacie: [HH:mm:ss].

Parameters:

msg - Pełny tekst logu.

Returns:

Tekst zawierający znacznik czasu lub pusty ciąg.

3.3.5. *private static String extractNodeId1(String msg)*

Wydobywa identyfikator pierwszego komputera (nadawcy) z logu.

Parameters:

msg - Pełny tekst logu.

Returns:

Fragment tekstu z identyfikatorem komputera lub pusty ciąg.

3.3.6. *private static String extractRest1(String msg)*

Wydobywa część tekstu pomiędzy pierwszym a drugim komputerem w logu.

Parameters:

msg - Pełny tekst logu.

Returns:

Fragment tekstu pomiędzy nazwami komputerów.

3.3.7. *private static String extractNodeId2(String msg)*

Wydobywa identyfikator drugiego komputera (odbiorcy) z logu.

Parameters:

msg - Pełny tekst logu.

Returns:

Fragment tekstu z identyfikatorem odbiorcy lub pusty ciąg.

3.3.8. *private static String extractBits*

Wydobywa dane binarne (ciąg bitów) z logu.

Parameters:

msg - Pełny tekst logu.

Returns:

Dane binarne w formie tekstowej lub pusty ciąg.

3.3.9. *private static String extractRest2(String msg)*

Zwraca pozostały fragment wiadomości po usunięciu czasu, komputerów i bitów.

Parameters:

msg - Pełny tekst logu.

Returns:

Reszta wiadomości po oczyszczeniu.

3.3.10. *public static void clearLog()*

Czyści panel logów, usuwając wszystkie dotychczasowe wpisy.

3.4. Class – MainApp

Główna klasa uruchamiająca aplikację symulacji sieci komputerowej. Inicjalizuje kontroler symulacji, buduje okno główne (GUI) oraz uruchamia proces symulacji. Klasa dziedziczy po Application, dzięki czemu może być bezpośrednio uruchamiana jako aplikacja JavaFX.

3.4.1. *Field Details*

- **controller**

[SimulationController](#) controller

Główny kontroler zarządzający logiką symulacji i stanem sieci.

3.4.2. *public void start(Stage stage)*

Metoda startowa aplikacji JavaFX. Tworzy i inicjalizuje obiekty niezbędne do działania symulacji:

- kontroler symulacji,
- strukturę sieci (węzły i połączenia),
- główne okno interfejsu graficznego,
- scenę i główne okno aplikacji.

Specified by:

start in class javafx.application.Application

Parameters:

stage - Główna scena aplikacji JavaFX.

3.4.3. *public static void main(String[] args)*

Punkt wejścia programu. Uruchamia aplikację JavaFX.

Parameters:

args - Argumenty przekazywane do programu z linii komend.

3.5. Class – MainWindow

Główne okno interfejsu graficznego aplikacji symulacyjnej. Łączy trzy główne panele interfejsu:

- [GraphPanel](#) – wizualizacja sieci i transmisji danych,
- [ControlPanel](#) – narzędzia do zarządzania symulacją i węzłami,
- [LogPanel](#) – panel wyświetlający logi systemowe.

3.5.1. Field Details

- ***graphPanel***

```
private GraphPanel graphPanel
```

Panel odpowiedzialny za graficzną prezentację sieci komputerowej.

- ***controlPanel***

```
private ControlPanel controlPanel
```

Panel sterowania symulacją.

- ***logPanel***

```
private LogPanel logPanel
```

Panel wyświetlający logi systemowe z przebiegu symulacji.

3.5.2. ***public MainWindow(SimulationController controller)***

Tworzy główne okno aplikacji oraz inicjalizuje panele interfejsu.

Parameters:

controller - Główny kontroler symulacji odpowiedzialny za logikę działania sieci.

3.5.3. ***public GraphPanel getGraphPanel()***

Zwraca referencję do panelu wizualizacji grafu sieci.

Returns:

Obiekt [GraphPanel](#) aktualnie wyświetlany w oknie.

4. Package – pob.pob_project.network

4.1. Class – Connection

Reprezentuje połączenie pomiędzy dwoma węzłami ([Node](#)) w grafie sieci. Obiekt tej klasy odpowiada połączeniu w topologii sieci, pozwalającej na przesyłanie pakietów pomiędzy dwoma węzłami.

4.1.1. Field Details

- **nodeA**

```
private final Node nodeA
```

Pierwszy węzeł połączenia.

- **nodeB**

```
private final Node nodeB
```

Drugi węzeł połączenia.

4.1.2. **public Connection(Node a, Node b)**

Tworzy połączenie pomiędzy dwoma węzłami sieci.

Parameters:

a - Pierwszy węzeł połączenia.

b - Drugi węzeł połączenia.

4.1.3. **public Node getNodeA()**

Zwraca pierwszy węzeł połączenia.

Returns:

Obiekt [Node](#) będący pierwszym końcem połączenia.

4.1.4. **public Node getNodeB()**

Zwraca drugi węzeł połączenia.

Returns:

Obiekt [Node](#) będący drugim końcem połączenia.

4.2. Class – NetworkGraph

Reprezentuje strukturę grafu sieciowego używaną w symulacji. Graf składa się z węzłów ([Node](#)) połączonych ze sobą ([Connection](#)). Klasa umożliwia dodawanie nowych węzłów oraz tworzenie połączeń między nimi, które są dwukierunkowe.

4.2.1. Field Details

- **nodes**

```
private final List<Node> nodes
```

Lista wszystkich węzłów w grafie.

- **connections**

```
private final List<Connection> connections
```

Lista wszystkich połączeń (krawędzi) pomiędzy węzłami.

4.2.2. ***public void addNode(Node n)***

Dodaje nowy węzeł do grafu sieciowego.

Parameters:

n - Węzeł, który ma zostać dodany do grafu.

4.2.3. ***public void addConnection(Node a, Node b)***

Tworzy dwukierunkowe połączenie pomiędzy dwoma węzłami. Połączenie jest rejestrowane w grafie oraz aktualizuje listy sąsiadów w obu węzłach.

Parameters:

a - Pierwszy węzeł połączenia.

b - Drugi węzeł połączenia.

4.2.4. ***public List<Node> getNodes()***

Zwraca listę wszystkich węzłów w grafie.

Returns:

Lista obiektów `Node`.

4.2.5. ***public List<Connection> getConnections()***

Zwraca listę wszystkich połączeń w grafie.

Returns:

Lista obiektów `Connection`.

4.2.6. ***public List<Node> getNeighbors(Node n)***

Zwraca listę sąsiadów dla danego węzła. Jeśli przekazany węzeł jest wartością null, zwracana jest pusta lista.

Parameters:

n - Węzeł, którego sąsiedzi mają zostać zwrócenii.

Returns:

Lista sąsiadów lub pusta lista, jeśli węzeł nie istnieje.

4.3. ***Class – Node***

Reprezentuje pojedynczy komputer (węzeł) w symulowanej sieci. Każdy węzeł działa w osobnym wątku, obsługując kolejkę przychodzących pakietów. Może wysyłać dane, odbierać pakiety, wykrywać błędy CRC oraz symulować różne typy usterek.

4.3.1. *Field Details*

- ***id***

private final int id

Identyfikator węzła w sieci.

- ***isActive***

private boolean isActive

Czy węzeł jest aktywny (może wysyłać i odbierać dane).

- ***incomingQueue***

private final [BlockingQueue](#)<[Packet](#)> incomingQueue

Kolejka pakietów oczekujących na przetworzenie.

- ***neighbors***

private final [List](#)<[Node](#)> neighbors

Lista sąsiadów (węzłów bezpośrednio połączonych z tym).

- ***currentFault***

private [Fault](#) currentFault

Aktualna usterka (jeśli występuje).

- ***polynomial***

private [String](#) polynomial

Wielomian używany do obliczania CRC.

- ***crcUtil***

private final [CRCUtil](#) crcUtil

Narzędzie do generowania i weryfikacji CRC.

- ***nodeThread***

```
private Thread nodeThread
```

Wątek obsługujący działanie węzła.

- ***controller***

```
private final SimulationController controller
```

Referencja do kontrolera symulacji.

- ***lastSentPackets***

```
private final Map<Integer,Packet> lastSentPackets
```

Ostatnio wysłane pakiety, używane do retransmisji po błędzie.

- ***sentCount***

```
private int sentCount
```

Liczba wysłanych pakietów.

- ***receivedCount***

```
private int receivedCount
```

Liczba odebranych pakietów.

- ***errorCount***

```
private int errorCount
```

Liczba błędów wykrytych przez ten węzeł.

4.3.2. *public Node(int id, SimulationController controller)*

Tworzy nowy węzeł o podanym identyfikatorze i łączy go z kontrolerem symulacji.

Parameters:

id - Unikalny identyfikator węzła.

controller - Kontroler zarządzający całą symulacją.

4.3.3. *public void addNeighbor(Node n)*

Dodaje sąsiada (połączenie dwukierunkowe w grafie).

4.3.4. *public void start()*

Uruchamia wątek węzła i rozpoczyna jego działanie.

4.3.5. *public void sendData(Node target, String message)*

Wysyła pakiet danych do innego węzła. Uwzględnia błędy, takie jak: błędy CRC, opóźnienia, utraty pakietów czy odwrócenie bitu.

Parameters:

target - Węzeł docelowy (odbiorca).

message - Treść wiadomości do wysłania.

4.3.6. *public void receivePacket(Packet packet)*

Odbiera pakiet i wykonuje odpowiednie działania w zależności od jego typu.

- Jeśli to pakiet ACK – potwierdza lub żąda retransmisji.
- Jeśli to pakiet danych – weryfikuje poprawność CRC.

Parameters:

packet - Pakiet do przetworzenia.

Throws:

[InterruptedException](#) - W przypadku przerwania wątku podczas oczekiwania.

4.3.7. *public void run()*

Główna pętla wątku węzła. Oczekuje na pakiety w kolejce i przetwarza je sekwencyjnie.

Specified by:

[run](#) in interface [Runnable](#)

4.3.8. *private Node findNeighborById(int id)*

Wyszukuje sąsiada po identyfikatorze.

4.3.9. *private String flipRandomBit(String data, String polynomial)*

Wykonuje losową inwersję bitu w danych, symulując błąd transmisji. W tym przypadku zmienia tylko bity końcowe (CRC), aby pokazać działanie w logach.

4.3.10. *public void injectFault(ErrorType type)*

Wstrzykuje błąd określonego typu do węzła. Zmienia jego stan.

Parameters:

type - Typ błędu do wstrzyknięcia.

4.3.11. *public void repairFault()*

Naprawia aktualny błąd i przywraca normalne działanie węzła. Aktualizuje stan wizualny w interfejsie.

4.3.12. *public void setCrcPolynomial(String polynomial)*

Ustawia wielomian używany do obliczania CRC.

4.3.13. *private void updateStatsInUI()*

Aktualizuje dane statystyczne w interfejsie użytkownika.

4.3.14. *public int getId()*

Returns:

Identyfikator węzła.

4.3.15. *public BlockingQueue<Packet> getQueue()*

Returns:

Kolejka przychodzących pakietów.

4.3.16. *public boolean isActive()*

Returns:

Czy węzeł jest aktywny.

4.3.17. *public List<Node> getNeighbors()*

Returns:

Lista sąsiadów (połączonych węzłów).

4.3.18. *public Fault getCurrentFault()*

Returns:

Aktualny błąd (usterka), jeśli występuje.

4.3.19. *public String getCurrentErrorText()*

Returns:

Tekstowy opis aktualnego błędu lub "brak" jeśli nieaktywny.

4.3.20. *public int getErrorCount()*

Returns:

Liczba wykrytych błędów.

4.3.21. *public int getSentCount()*

Returns:

Liczba wysłanych pakietów.

4.3.22. *public int getReceivedCount()*

Returns:

Liczba odebranych pakietów.

4.4. Class – Packet

Reprezentuje pakiet danych przesyłany pomiędzy komputerami (węzłami) w sieci. Pakiet może zawierać dane użytkownika lub być pakietem potwierdzenia (ACK). Dodatkowo może posiadać opóźnienie, co pozwala symulować warunki sieciowe, takie jak spowolnienie transmisji.

4.4.1. Field Details

- **data**

private [String](#) data

Dane zawarte w pakiecie (ciąg bitów lub tekst).

- ***sourceId***

```
private final int sourceId
```

Identyfikator źródła (nadawcy pakietu).

- ***destinationId***

```
private final int destinationId
```

Identyfikator miejsca docelowego (odbiorcy pakietu).

- ***isDelayed***

```
private boolean isDelayed
```

Czy pakiet jest aktualnie opóźniony w transmisji.

- ***delay***

```
private int delay
```

Czas opóźnienia pakietu w milisekundach.

- ***isAck***

```
private boolean isAck
```

Czy pakiet jest potwierdzeniem (ACK).

- ***ackPositive***

```
private boolean ackPositive
```

Typ potwierdzenia: true – ACK pozytywny, false – żądanie retransmisji (NACK).

- ***random***

```
private static final Random random
```

Generator losowy, potencjalnie wykorzystywany do symulacji losowych zjawisk.

4.4.2. *public Packet(String data, int sourceId, int destinationId)*

Tworzy nowy pakiet z danymi przesyłanymi między dwoma węzłami.

Parameters:

data - Zawartość pakietu (np. dane lub wiadomość).

sourceId - Identyfikator węzła nadawcy.

destinationId - Identyfikator węzła odbiorcy.

4.4.3. *public static Packet createAckPacket(int sourceId, int destinationId, boolean ackPositive)*

Tworzy specjalny pakiet typu ACK (potwierdzenie lub NACK).

Parameters:

sourceId - Identyfikator węzła wysyłającego potwierdzenie.

destinationId - Identyfikator odbiorcy potwierdzenia.

ackPositive - Wartość true - ACK pozytywny, false - żądanie retransmisji.

Returns:

Nowy pakiet potwierdzający.

4.4.4. *public String getData()*

Returns:

Dane zawarte w pakiecie.

4.4.5. *public int getSourceId()*

Returns:

Identyfikator węzła nadawcy.

4.4.6. *public int getDestinationId()*

Returns:

Identyfikator węzła odbiorcy.

4.4.7. *public boolean isDelayed()*

Returns:

Czy pakiet jest aktualnie opóźniony.

4.4.8. *public void setIsDelayed(boolean isDelayed)*

Ustawia, czy pakiet ma być opóźniony w transmisji.

Parameters:

isDelayed - Wartość true, jeśli pakiet powinien być opóźniony.

4.4.9. *public boolean isAck()*

Returns:

Czy pakiet jest pakietem typu ACK.

4.4.10. *public boolean isAckPositive()*

Returns:

Czy ACK jest true czy false.

4.4.11. *public void setAck(boolean ack)*

Ustawia typ pakietu (ACK lub zwykły).

4.4.12. *public void setAckPositive(boolean ackPositive)*

Ustawia typ potwierdzenia ACK: true lub false.

4.4.13. *public void setDelay(int delay)*

Ustawia opóźnienie pakietu w milisekundach.

Parameters:

delay - Czas opóźnienia w ms.

4.4.14. *public int getDelay()*

Returns:

Aktualne opóźnienie pakietu w milisekundach.

5. Package – *pob.pob_project.simulation*

5.1. Class - *Logger*

Klasa odpowiedzialna za logowanie zdarzeń w systemie symulacji. Obsługuje zarówno wypisywanie logów w konsoli, jak i przekazywanie ich do panelu interfejsu graficznego [LogPanel](#). Każdy wpis logu jest automatycznie opatrzony znacznikiem czasu w formacie HH:mm:ss:SSS.

5.1.1. *Field Details*

- ***sdf***

```
private static final SimpleDateFormat sdf
```

Format daty używany w znacznikach czasu logów.

5.1.2. *public static synchronized void log(String message)*

Dodaje komunikat do logu systemowego oraz wyświetla go w konsoli i panelu GUI. Metoda jest zsynchronizowana, aby zapewnić bezpieczeństwo wątkowe podczas logowania z wielu wątków symulacji.

Parameters:

message - Treść komunikatu do zapisania w logu.

5.2. Class – *SimulationController*

Klasa nadrzędna sterująca przebiegiem symulacji sieci. Odpowiada za inicjalizację topologii sieci (tworzenie węzłów i połączeń), uruchamianie procesów symulacyjnych, konfigurację wielomianu CRC oraz obsługę błędów wstrzykiwanych do węzłów.

5.2.1. *Field Details*

- ***graph***

```
private final NetworkGraph graph
```

Reprezentacja grafu sieciowego składającego się z węzłów i połączeń.

- ***errorInjector***

```
private final ErrorInjector errorInjector
```

Moduł odpowiedzialny za wstrzykiwanie błędów do węzłów.

- ***crcPolynomial***

```
private String crcPolynomial
```

Aktualnie ustawiony wielomian CRC w formacie binarnym (np. "1010").

- **graphPanel**

private [GraphPanel](#) graphPanel

Odwołanie do panelu graficznego, wizualizującego sieć i statystyki.

5.2.2. *public SimulationController()*

Tworzy nową instancję kontrolera symulacji.

5.2.3. *public void initializeNetwork()*

Inicjalizuje sieć poprzez utworzenie 10 węzłów i zdefiniowanie podstawowych połączeń między nimi.

5.2.4. *public void startSimulation()*

Uruchamia symulację, aktywując wszystkie węzły w sieci oraz ustawiając domyślny wielomian CRC.

5.2.5. *public boolean checkCrcPolynomial(String polynomial)*

Weryfikuje poprawność formatu wielomianu CRC.

Parameters:

polynomial - Ciąg bitów reprezentujący wielomian CRC.

Returns:

Wartość true Jeśli wielomian jest poprawny (ma długość 2–32 bity, zawiera tylko zera i jedynki oraz zaczyna się od '1').

5.2.6. *public boolean checkNeighbors(Node src, Node dst)*

Sprawdza, czy dwa węzły są bezpośrednimi sąsiadami w grafie.

Parameters:

src - Węzeł źródłowy.

dst - Węzeł docelowy.

Returns:

Wartość true jeśli węzły są połączone, w przeciwnym razie false.

5.2.7. *public void injectError(Node node, ErrorType type)*

Wstrzykuje określony typ błędu do wybranego węzła sieci.

Parameters:

node - Węzeł, do którego zostanie wprowadzony błąd.

type - Typ błędu ([ErrorType](#)).

5.2.8. *public void repairNode(Node node)*

Naprawia awarię w wybranym węźle (usuwa aktualny błąd).

Parameters:

node - Węzeł, w którym zostanie usunięta usterka.

5.2.9. *public NetworkGraph getGraph()*

Returns:

Graf symulacji.

5.2.10. *public String getCrcPolynomial()*

Returns:

Aktualnie używany wielomian CRC.

5.2.11. *public void setCrcPolynomial(String newPoly)*

Ustawia nowy wielomian CRC i propaguje go do wszystkich węzłów sieci.

Parameters:

newPoly - Nowy wielomian CRC (ciąg bitów).

5.2.12. *public void setGraphPanel(GraphPanel panel)*

Ustawia panel graficzny odpowiedzialny za wizualizację sieci.

5.2.13. *public GraphPanel getGraphPanel()*

Returns:

Panel graficzny aktualnie używany przez kontroler.